Business Intelligence Lab

Using the Kaggle Superstore Dataset

Python | pandas | matplotlib

Learning Objectives

By the end of this lab, students will be able to:

- Load a real-world CSV dataset from Kaggle using pandas
- Explore data structure using shape, dtypes, and describe()
- Clean data and add calculated columns (e.g. Profit Margin %)
- Use groupby() to summarise data by category and region
- Create pivot tables using pivot_table() for cross-tabulation
- Build a bar chart and pie chart using matplotlib
- Interpret results and write a short business insight report

Dataset Overview — Superstore

The Superstore dataset is one of the most widely used beginner BI datasets. It contains retail order data from a fictional US superstore covering 2014–2017.

9,994	21	3 Categories	4 Regions
Rows	Columns	Furniture, Office Supplies, Technology	North, South, East, West

Key Columns Used in This Lab

Column	Description
Category	Product type: Furniture, Office Supplies, Technology
Sub-Category	Product detail (e.g. Chairs, Phones, Paper)
Region	Sales region: North, South, East, West
Sales	Total revenue per order (float)
Profit	Net profit per order (float, can be negative)
Quantity	Number of units sold (integer)
Order Date	Date of order — convert to datetime in Python
Segment	Customer type: Consumer, Corporate, Home Office

1 Setup and Load the Kaggle Dataset

Duration: 0 – 12 minutes

Method A — Google Colab (Recommended for Beginners)

Step 1 Create a Google Colab notebook

Go to colab.research.google.com and click New Notebook. This runs Python in your browser with no installation required.

Step 2 Download the dataset from Kaggle

Visit the Kaggle dataset URL listed above. Click the Download button and save the file superstore.csv to your computer. A free Kaggle account is required.

Step 3 Upload the file to Colab

In your Colab notebook, run this cell to upload the file:

```
from google.colab import files
uploaded = files.upload()   # click Choose Files and select superstore.csv
```

Step 4 Import libraries and load the data

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('superstore.csv', encoding='latin-1')
print('Shape:', df.shape)
df.head()
```

Tip: encoding='latin-1' is required — the CSV uses special characters that the default UTF-8 encoding cannot read. Always check encoding when loading real-world data files.

Method B — Local Jupyter (with Kaggle API)

Step 1 Get your Kaggle API key

Go to kaggle.com -> Account -> Create API Token. This downloads a file called kaggle.json. Place it in the ~/.kaggle/ folder on your computer.

Step 2 Install kaggle and download the dataset

```
pip install kaggle
kaggle datasets download -d vivek468/superstore-dataset-final --unzip
```

Step 3 Load the file

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('Sample - Superstore.csv', encoding='latin-1')
```

```
print('Shape:', df.shape)
```

Note: The exact filename downloaded may be 'Sample - Superstore.csv' — check what was saved and update the path if needed.

2 Explore and Clean the Data

Duration: 12 – 22 minutes

Step 1 Understand the dataset structure

Run each of these commands and read what they tell you:

```
# How many rows and columns?
print('Shape:', df.shape)

# Column names and data types
print(df.dtypes)

# Check for missing values
print('Missing values:')
print(df.isnull().sum())

# Descriptive statistics
df[['Sales', 'Profit', 'Quantity']].describe().round(2)
```

Expected output (approximate):

```
          Sales     Profit  Quantity
count  9994.00    9994.00   9994.00
mean    229.86      28.66      3.79
min       0.44   -6599.98      1.00
max   22638.48    8399.98     14.00
```

Tip: This dataset is very clean — isnull().sum() should show all zeros. In real BI projects, missing values are common and must be handled before analysis.

Step 2 Parse the Order Date column

Convert the date column from text to a proper datetime so you can analyse trends by month or year:

```
# Convert Order Date to datetime
df['Order Date'] = pd.to_datetime(df['Order Date'])

# Extract year and month as new columns
df['Year']  = df['Order Date'].dt.year
df['Month'] = df['Order Date'].dt.month

print(df['Year'].value_counts().sort_index())
```

Tip: The dataset covers 2014-2017. You can filter to one year with: df = df[df['Year'] == 2017]

Step 3 Add a Profit Margin calculated column

Profit margin % shows how profitable each sale is — a fundamental BI metric:

```
# Profit Margin % = (Profit / Sales) x 100
df['Profit Margin %'] = (df['Profit'] / df['Sales'] * 100).round(2)

print(df[['Category', 'Sales', 'Profit', 'Profit Margin %']].head(5))
```

3 Analyse with groupby() and pivot_table()

Duration: 22 – 37 minutes

Step 1 Sales and Profit by Category

```
# Total Sales and Profit by Category
cat_summary = df.groupby('Category')[['Sales', 'Profit']].sum().round(2)
cat_summary['Margin %'] = (cat_summary['Profit'] / cat_summary['Sales'] *
100).round(1)
print(cat_summary.sort_values('Sales', ascending=False))
```

Expected output:

```
                  Sales     Profit  Margin %
Category
Technology       836154.03  145454.95    17.4
Furniture        741999.80   18451.27     2.5
Office Supplies  719047.03  122490.80    17.0
```

Tip: Technology has the highest sales AND profit margin. Furniture sells well but has a very thin margin — this is a real business insight worth discussing!

Step 2 Sales by Region

```
# Total Sales by Region
reg_sales = df.groupby('Region')['Sales'].sum().sort_values(ascending=False)
print(reg_sales.round(2))

# Share as percentage
print('\nShare %:')
print((reg_sales / reg_sales.sum() * 100).round(1))
```

Step 3 Region x Category Pivot Table

```
# Pivot table: Region vs Category (Sales)
pivot = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Category',
    aggfunc='sum',
    fill_value=0
).round(2)
print(pivot)
```

Note: fill_value=0 replaces NaN with 0 for any region/category combinations with no sales. Always do this to avoid misleading blank cells in BI reports.

Step 4 Top 10 Most Profitable Sub-Categories

```
# Top 10 sub-categories by total profit
top10 = df.groupby('Sub-Category')['Profit'].sum()\
          .sort_values(ascending=False).head(10)
print(top10.round(2))
```

4 Build BI Charts with matplotlib

Duration: 37 – 52 minutes

Step 1 Bar Chart — Sales vs Profit by Category

```
fig, ax = plt.subplots(figsize=(9, 5))

x = range(len(cat_summary))
width = 0.35

bars1 = ax.bar([i - width/2 for i in x], cat_summary['Sales'],
               width, label='Sales', color='#378ADD')
bars2 = ax.bar([i + width/2 for i in x], cat_summary['Profit'],
               width, label='Profit', color='#1D9E75')

ax.set_xticks(list(x))
ax.set_xticklabels(cat_summary.index, fontsize=12)
ax.set_title('Sales vs Profit by Category', fontsize=14, fontweight='bold')
ax.set_ylabel('Amount ($)')
ax.legend()
ax.yaxis.set_major_formatter(
    plt.FuncFormatter(lambda x, _: f'${x/1000:.0f}K'))

plt.tight_layout()
plt.savefig('chart_category.png', dpi=150)
plt.show()
```

Step 2 Pie Chart — Sales Share by Region

```
colors = ['#378ADD', '#1D9E75', '#EF9F27', '#D85A30']

plt.figure(figsize=(7, 7))
plt.pie(reg_sales.values, labels=reg_sales.index,
        colors=colors, autopct='%1.1f%%',
        startangle=140, pctdistance=0.75)
plt.title('Sales Share by Region', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('chart_region.png', dpi=150)
plt.show()
```

Step 3 Challenge — Monthly Sales Trend (Line Chart)

If you finish early, create a line chart to identify seasonal trends:

```
# Line chart: Monthly sales trend for 2017
df17 = df[df['Year'] == 2017]
monthly = df17.groupby('Month')['Sales'].sum()

plt.figure(figsize=(10, 5))
plt.plot(monthly.index, monthly.values,
         marker='o', linewidth=2, color='#4f46e5')
```

```
plt.fill_between(monthly.index, monthly.values,
                 alpha=0.1, color='#4f46e5')
plt.xticks(range(1,13),
    ['Jan','Feb','Mar','Apr','May','Jun',
     'Jul','Aug','Sep','Oct','Nov','Dec'])
plt.title('Monthly Sales Trend - 2017', fontsize=14, fontweight='bold')
plt.ylabel('Sales ($)')
plt.tight_layout()
plt.show()
```

5 Auto BI Report and Reflection

Duration: 52 – 60 minutes

Step 1 Print an Automated BI Insight Report

Run this code to auto-generate a written summary from your data — similar to a real BI dashboard report:

```
top_cat     = cat_summary['Sales'].idxmax()
top_profit  = cat_summary['Profit'].idxmax()
low_margin  = cat_summary['Margin %'].idxmin()
top_region  = reg_sales.idxmax()
total_sales  = df['Sales'].sum()
total_profit = df['Profit'].sum()

print(f'''
========================================
       SUPERSTORE BI REPORT
========================================
Total Sales        : ${total_sales:,.0f}
Total Profit       : ${total_profit:,.0f}
Overall Margin     : {total_profit/total_sales*100:.1f}%

Top Sales Category : {top_cat}
Most Profitable    : {top_profit}
Lowest Margin      : {low_margin}  <- needs attention
Best Region        : {top_region}

KEY INSIGHT:
{low_margin} generates good sales volume but has the
lowest profit margin. Review pricing or costs.
========================================
''')
```

Step 2 Write Your Reflection

Add a text cell (Colab) or comment block at the bottom of your script with answers to all five questions below:

```
"""
REFLECTION — Student Name: _______________
Date: _______________

1. Which category has the highest total sales?
   Answer:

2. Which category has the LOWEST profit margin — and
```

```
   why might that be a concern for the business?
   Answer:

3. Which region contributes the most revenue?
   Answer:

4. What recommendation would you give the store manager
   based on your charts and analysis?
   Answer:

5. What additional data would improve this analysis?
   (e.g. customer age, marketing spend, store location)
   Answer:
"""
```

Step 3 Save and Submit

Save your file as YourName_BI_Superstore.ipynb (or .py). Your submission must include:

- The Python script or Colab notebook with all code cells run
- chart_category.png — the bar chart
- chart_region.png — the pie chart
- Reflection answers filled in

> **Tip:** In Colab: File -> Download -> Download .ipynb. Chart PNG files appear in the Files panel (left sidebar) — right-click to download each one.

Lab Completion Checklist

Tick each item as you complete it during the lab:

- □ Downloaded superstore.csv from Kaggle
- □ Loaded CSV with correct encoding (latin-1)
- □ Checked shape, dtypes, and missing values using pandas
- □ Parsed Order Date column and extracted Year and Month
- □ Added Profit Margin % calculated column
- □ Used groupby() for Category and Region summaries
- □ Created pivot_table() for Region x Category matrix
- □ Found the top 10 most profitable sub-categories
- □ Built a labeled Sales vs Profit bar chart (saved as PNG)
- □ Built a pie chart for regional sales share (saved as PNG)
- □ Printed the automated BI insight report
- □ Answered all 5 reflection questions
- □ Saved and submitted .ipynb file and chart PNGs

Quick Reference — Key Python Commands

Command	What It Does
pd.read_csv('file.csv', encoding='latin-1')	Load a CSV file into a DataFrame
df.shape	Returns (rows, columns)
df.dtypes	Shows data type of each column
df.isnull().sum()	Counts missing values per column
df.describe()	Summary statistics (mean, min, max, etc.)
df['col'] = df['a'] * df['b']	Create a calculated column
pd.to_datetime(df['col'])	Convert text to datetime format
df.groupby('col')['val'].sum()	Sum values grouped by a category
df.pivot_table(values, index, columns, aggfunc)	Create a cross-tabulation matrix
plt.bar(x, y)	Create a bar chart
plt.pie(values, labels, autopct)	Create a pie chart
plt.savefig('name.png', dpi=150)	Save chart as image file